Assignment 4

Web Attacks: SQL, XSS, CSRF

Network Security (50.020)

0. Objectives

In this lab, you will learn:

- Common web application vulnerabilities and their security impacts.
- SQL injection attacks and detection.
- Cross-Site Scripting (XSS) attacks and detection.
- Cross-Site Request Forgery (CSRF) attacks and detection.
- Database operations, information disclosure, and data manipulation.
- AI-assisted web security attack detection tools and techniques.

Lab Setup:

- VirtualBox
- 1 Kali VM (root:toor) + NAT Network

1. Web Application Setup

The Example Campus Service portal provides an integrated platform for managing campus service tickets, staff profiles, knowledge-based articles, access requests, and account settings. The system supports multiple user accounts with different access privileges and includes a separate document sharing service. In this lab, the web application will be deployed directly on the Kali Linux VM using Apache, PHP, and **MariaDB**.

Copy **lab4_setup.zip** to the Kali Desktop, open a terminal, and execute:

```
cd ~/Desktop
unzip lab4_setup.zip
cd Example-Campus-Service-Lab
```

Start the web service:

```
chmod +x *
./start.sh
```

Now you can access the web application using Firefox:

- Campus Service Portal: http://portal.campus.test/
- Document Sharing Service: http://docs.campus.test:8082/

Note: Running ./restart.sh restores the initial database and application files. It will reset the web application to the initial state.

2.	**SQL Injection** SQL Injection (SQLi) occurs when untrusted user input is incorporated into SQL queries without proper validation or parameterization. Attackers can manipulate these queries to bypass authentication, retrieve sensitive information, or modify database records. In this portal, we have default users: ➢ admin:admin ➢ alice:alice ➢ bob:bob Use SQL injection on the login page, click Sign in, and observe the result. Consider how the injected SQL condition affects the authentication process. Can you log in as Admin? • ' or 1=1 # Alice is a campus staff member who uses the **Service Tickets** system to manage service requests. Some tickets are marked as private and should only be accessible to their owners. Alice wants to investigate Bob's private service tickets and determine whether the search function can be exploited to access restricted database information. The Service Tickets search function is a potential attack point because it accepts user-supplied keywords and uses them to query the database. In a real attack, the attacker typically does not know the database name, table names, or column names beforehand. This information must be discovered progressively. Log in as alice, navigate to **Service Tickets**, and complete the following investigation: Explore the search function using different inputs, including special characters such as '. Observe the results and examine any SQL error messages for clues about the underlying query. The attacker will usually discover the database name first. Investigate how the UNION SELECT operator can display additional query results. Try the following input in the Service Tickets search field: • zzzx%' UNION SELECT 9999,DATABASE(),'Open','low','private','database' # Observe the result and identify the current database name. Consider why the injected SELECT statement requires six values. Can you use AI to get this SQL command if you are the attacker? Now that you know the database name, investigate which tables exist. You may need to try multiple times to get the structure of the database. **Question: If Alice is the attacker, can she reveal Bob's public and private list data? Can she retrieve all usernames and password hashes? Can she modify the admin's password?** Demonstrate the complete SQL injection attack through the Service Tickets search function, from identifying the vulnerability and discovering the database structure to retrieving Bob's private tickets and user password hashes. Then, demonstrate how to modify the administrator's password to abc using SQL injection. Include SQL inputs used and relevant screenshots in the report. Explain how secure coding practices could prevent these attacks.

3. Cross-Site Scripting (XSS)

Cross-Site Scripting (XSS) is a web application vulnerability that allows attackers to inject JavaScript code into webpages viewed by other users. When the application fails to properly handle untrusted input, the injected code may execute in the victim's browser with the privileges of the affected website. Attackers can exploit XSS to manipulate webpage content, access readable session cookies, or perform unauthorized actions. In this exercise, you will investigate reflected and stored XSS vulnerabilities in the Example Campus Service portal.

The portal provides a **Knowledge Base** search function that allows staff members to find articles and troubleshooting information.

Reflected XSS: Enter the following JavaScript code into the search field. Observe what happens when the search results are displayed.

- `<script>alert('XSS')</script>`

How to use XSS to get the session cookie? Examine the URL generated by the search and consider what would happen if another user opened the same URL.

Stored XSS: Alice wants to target other users through the Campus Service Portal. She discovers that staff members can edit their profiles, which are visible to other users.

You can navigate to the **Staff directory** for details**.** Alice attempts to store malicious JavaScript in her profile so that anyone visiting it will automatically execute the code in their own browser. For this exercise, your objective is to make sure your malicious JavaScript (Can just show the session cookie) will be activated whenever another user visits Alice's page.

Question: Compare reflected and stored XSS attacks, explain their potential security impacts, and describe how secure coding practices and browser security mechanisms can prevent or mitigate these attacks.

4. Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF) allows an attacker to trick an authenticated user's browser into performing unintended actions on a trusted website. The attack exploits the browser's automatic handling of session cookies and applications that fail to verify request legitimacy. CSRF attacks commonly involve specially crafted links, HTML elements, or forms that trigger unauthorized GET or POST requests.

Scenario: Alice has submitted an application to access a restricted campus laboratory, but only an administrator can approve it. Alice wants to exploit the administrator's active login session to approve her application without obtaining the administrator's credentials.

Log in as alice and navigate to Access requests. Locate AR-301 (After-hours Research Lab 2 access) and confirm its status is Pending.



	Try accessing directly as Alice. Observe the result.
	• http://portal.campus.test/request_action.php?id=301&decision=approve
	
	Open a separate Firefox Private Browsing window, log in as admin, and examine the approval function under Access requests. Do not approve the request yet. Return to Alice's browser and open the Document Sharing Service at
	• http://docs.campus.test:8082/
	
	Create a document using the following HTML template:
	
	`<!doctype html>`
	`<html lang="en">`
	`<head><meta charset="utf-8"><title>Hello</title></head>`
	`<body>`
	`<h1>Hello</h1>`
	`<p>Hi, how are you?</p>`
	`<p>I just approved my own request, surprise!.</p>`
	`<!-- Browser requests the URL as it loads this page. -->`
	`<img src="http://portal.campus.test/request_action.php?id=301&decision=approve" alt="" width="1" height="1" style="opacity:0">`
	`</body>`
	`</html>`
	
	Question: The attacker can create something like this. But how to trick the admin into clicking this? Show your idea. Then, return to Access requests and check whether AR-301 has changed from Pending to Approved. Explain why authentication and the Same-Origin Policy alone are insufficient to prevent these attacks.
	
	[Optional]:
	
	Scenario: Alice now wants to modify the administrator's account preferences so that security notifications are disabled and the notification delivery address is changed to her own email address. She prepares another document containing a hidden HTML form that submits a POST request when the administrator opens it. Demonstrate the POST-based CSRF attack used to modify the administrator's account settings. Include the completed HTML payload, observed results, and an explanation of how the forged POST request works.

5.	**AI-Assisted Web Attack Detection** As a security expert, you may need to protect legacy web applications with known vulnerabilities that cannot be immediately fixed. Use AI tools to assist you in **developing a real-time attack detection tool** capable of monitoring web activities and identifying SQL injection, XSS, and CSRF attacks performed in this lab. Demonstrate your tool by launching attacks and observing the generated security alerts. Explain your detection approach, design considerations, effectiveness, and limitations.
6.	**Submission** • 1 PDF Report: Include screenshots, answers, analysis results, and brief reflection on your AI experience and learning • Code Files: Scripts/Commands used in the lab.